Despliegue en Vercel

Preparación de Vercel

Actualizado el 23 de octubre de 2026. No hay proyecto Vercel ni base PostgreSQL remota configurados. Los cambios locales NO activan un dominio HTTPS externo. No se han enviado datos, certificados ni credenciales a Vercel.

Estado y condiciones antes de publicar

La configuración está preparada para el soporte nativo de NestJS en Vercel, con raíz backend y entrada src/main.ts. Flutter se compila localmente y se incluye en backend/public. No se ha validado todavía un despliegue real.

La auditoría de dependencias sigue teniendo avisos que requieren actualización y revisión; no abrir esta versión a clientes como si estuviera certificada. Mantener la primera instalación protegida y usar solo datos sintéticos.

Pasos en tu cuenta

1. Crear una cuenta Vercel y activar MFA. Crear el proyecto y mantener activada Deployment Protection en los entornos disponibles en tu plan; comprobar expresamente la protección de producción y no asumir que una URL oculta basta.

2. Crear PostgreSQL administrado con TLS, cifrado en reposo y backups. Verificar esas propiedades con el proveedor; este código no activa el cifrado del disco ni cifra cada columna. Usar bases separadas para pruebas y producción.

3. Configurar variables de entorno en Vercel, nunca dentro de Flutter:

Variable	Valor
NODE_ENV	production
JWT_SECRET	Secreto aleatorio, al menos 48 caracteres; generar 48 bytes aleatorios
DATABASE_URL	URL PostgreSQL del proveedor con sslmode=require&sslaccept=strict; pool compatible con Prisma 5
CORS_ORIGIN	Origen exacto https://nombre-del-proyecto.vercel.app o dominio propio, sin barra final
ALLOW_PUBLIC_REGISTRATION	false

Generar JWT_SECRET con node -e "console.log(require('crypto').randomBytes(48).toString('hex'))" en un terminal privado y guardarlo como variable sensible. No compartirlo por chat. Configurar el origen estable después de crear el proyecto. Cada preview necesita su origen autorizado y su propia base; no autorizar todos los subdominios Vercel.

4. Compilar Flutter desde sistema_tributario_rimpe:

```
flutter build web --release --dart-define=API_BASE_URL=
```

Desde la raíz local copiar el resultado:

```
Copy-Item -Path 'sistema_tributario_rimpe/build/web/*' -Destination 'backend/public' -Recurse -Force
```

5. Desde backend, con la URL remota configurada en un entorno privado de administración, ejecutar npm ci, npm run prisma:generate, npm run db:deploy y npm run seed. No ejecutar migraciones automáticamente en cada preview ni usar reset. Usar conexión directa para migraciones si el proveedor no permite hacerlo a través del pool; una cuenta de migración separada de la cuenta limitada de la aplicación es preferible.

6. Desplegar desde backend con la CLI oficial de Vercel: npx vercel. Vincularlo a tu proyecto, detectar NestJS y mantener vercel.json. .vercelignore excluye secretos y dependencias locales. Si usas Git en vez de CLI, public está ignorado por Git: necesitas incluir el artefacto Flutter en tu proceso de build; subir solo el código actual no construye Flutter.

7. Comprobar la URL HTTPS protegida: acceso, base, aislamiento por empresa, firma de prueba, descarga y encabezados. El backend rechaza HTTP en producción. Vercel proporciona TLS y certificados; no necesitas comprar un certificado.

8. Crear la primera cuenta únicamente en la instalación protegida: habilitar temporalmente ALLOW_PUBLIC_REGISTRATION=true, desplegar, registrarse y volver a false con otro despliegue. Verificar que register devuelve 403 antes de retirar cualquier protección. No hay creación automática de administrador.

Protecciones implementadas

- Secretos JWT obligatorios, algoritmo HS256 explícito y usuarios desactivados rechazados en cada petición autenticada.
- Renovación de token de un solo uso incluso con solicitudes simultáneas; contraseñas Argon2id y refresh tokens guardados como hash.
- Límite de intentos en PostgreSQL compartido por instancias, por IP y cuenta; identificadores del contador derivados por HMAC, sin correos/IP en claro allí.
- Registro cerrado por defecto en producción, contraseñas nuevas de mínimo 12 caracteres y límites de tamaño en las entradas de autenticación.
- Helmet, CSP, HSTS en producción, protección de marcos, no-sniff, política de permisos y caché desactivada en las respuestas API. Swagger cerrado en producción.
- Registros HTTP con identificador, ruta, método y estado, sin cuerpo, tokens, query string ni cabeceras sensibles. Auditoría redacta claves sensibles y XML en nuevos eventos; los registros históricos existentes no se han reescrito.
- Sesión web solo en memoria; al recargar hay que iniciar sesión otra vez. Elimina tokens guardados por versiones anteriores. Esto reduce persistencia, pero no elimina el riesgo de XSS mientras la página esté abierta.

Operación pendiente en el proveedor

- Activar firewall/rate limiting de Vercel antes de las funciones para reducir abuso que de otro modo llegaría a PostgreSQL. No está activado desde este chat.
- Configurar alertas sobre 401/403/429/500 y exportación de logs a almacenamiento con retención y permisos restringidos. Los logs de aplicación no son inmutables.
- Programar la eliminación de contadores expirados de security_rate_limits; fijar una política de conservación de auditoría y sesiones según necesidades.
- Backups cifrados, pruebas de restauración, rotación de secretos y revisión periódica de dependencias. No hay cifrado de extremo a extremo: el servidor necesita procesar XML y realizar la firma; los certificados no se persisten.

- Revisar MFA de usuarios de la aplicación y recuperación de cuenta antes de un despliegue público amplio. MFA en Vercel protege la administración del hosting, no sustituye MFA dentro de esta aplicación.

El ambiente productivo SRI sigue bloqueado; ver 04 Integracion SRI y alcance.docx. HTTPS no convierte los borradores ATS ni el módulo SRI de pruebas en un producto fiscal completo.

Referencias oficiales:

- https://vercel.com/docs/frameworks/backend/nestjs
- https://vercel.com/docs/cdn-security/encryption
- https://vercel.com/docs/environment-variables